AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 13.10.2026

Lab 02

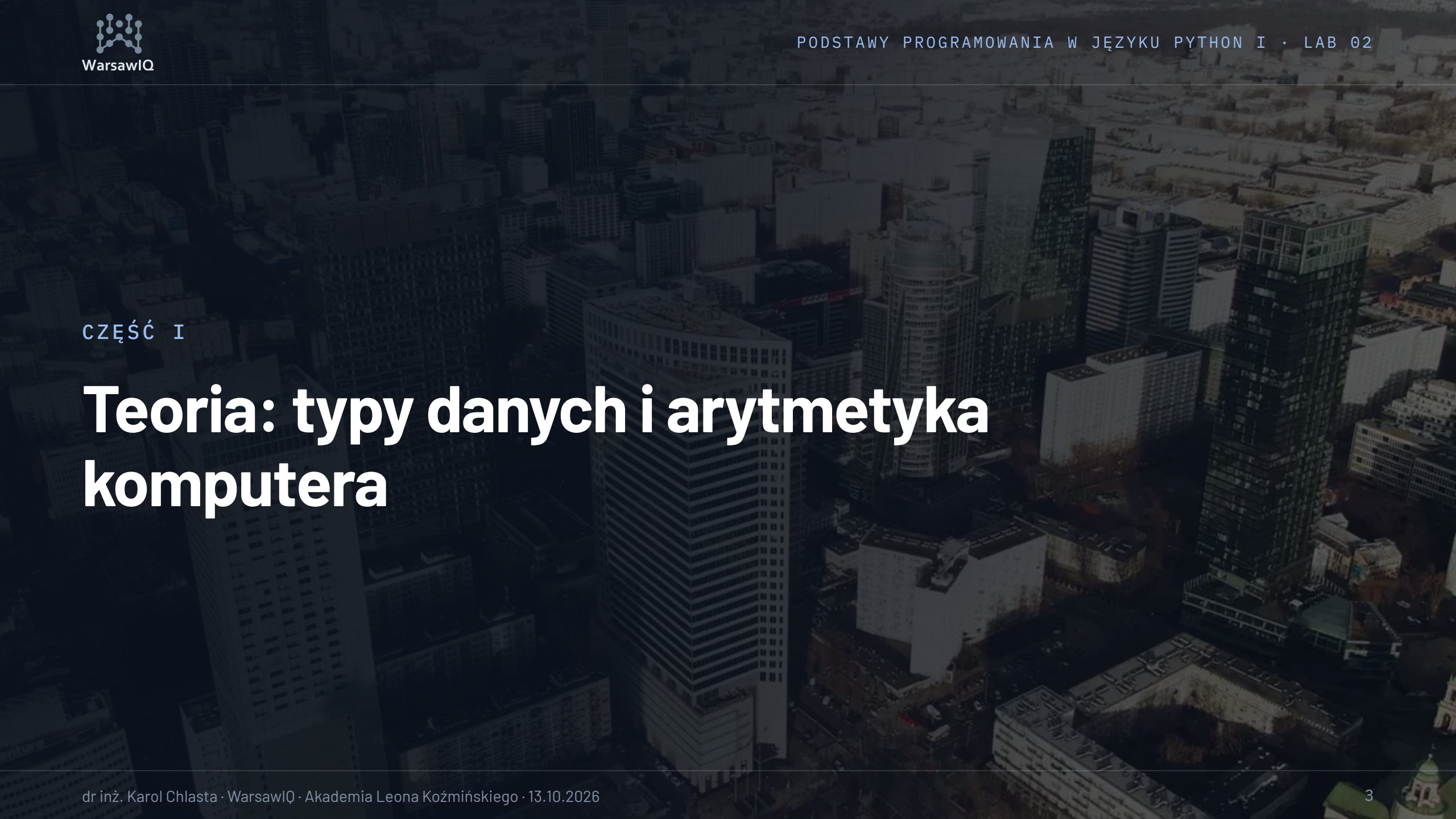
Zmienne, typy danych i operacje arytmetyczne

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć, **jak komputer przechowuje liczby** i skąd biorą się błędy obliczeń
- Typy `int`, `float`, `str`, `bool` i konwersja między nimi
- Operatory arytmetyczne `+ - * / // % **` i ich priorytety
- Przeliczanie jednostek: temperatura, waluty, czas
- Efekty z sylabusa: **U1**, **U3** (proste algorytmy), **KS1**



CZĘŚĆ I

Teoria: typy danych i arytmetyka komputera

Typ danych

> **Definicja.** Typ danych określa **zbiór wartości** oraz **operacje**, które wolno na nich wykonywać.

TYP	PRZYKŁAD	WARTOŚCI	ZMIENNY?
`int`	`42`	liczby całkowite, bez limitu	nie
`float`	`1.75`	liczby zmiennoprzecinkowe (IEEE 754)	nie
`complex`	`3+4j`	liczby zespolone	nie
`bool`	`True`	`True`, `False` (podtyp `int`)	nie
`str`	`"Anna"`	ciągi znaków Unicode	nie

- **Niezmienność** (*immutability*): `x = x + 1` tworzy **nowy obiekt**, nie zmienia starego
- `bool` to podtyp `int`: `True + True` → `2`

`int`: liczby całkowite bez limitu

- W Pythonie 3 `int` ma **dowolną precyzję**: rośnie, ile pozwala pamięć

```
2 ** 1000              # liczba z 302 cyframi
0b1010, 0o17, 0x1F     # (10, 15, 31) – zapis dwójkowy, ósemkowy, szesnastkowy
1_000_000              # podkreślenia dla czytelności (Python 3.6+)
bin(10)                # '0b1010'
```

- Dla porównania: w C i Javie `int` ma zwykle 32 bity, maks. **2 147 483 647**, a dalej następuje przepełnienie
- Od Pythona 3.11 zamiana `int` na tekst jest ograniczona do **4300 cyfr** (ochrona przed atakami DoS, CVE-2020-10735):

```
str(10 ** 5000)   # ValueError: Exceeds the limit (4300 digits)...
```

`float` : standard IEEE 754

Liczba zmiennoprzecinkowa to zapis „naukowy” w systemie dwójkowym:

$$x = (-1)^s \cdot 1{,}m \cdot 2^e$$

POLE (BINARY64)	BITY
znak *s*	1
wykładnik *e*	11
mantysa *m*	52 (+1 bit ukryty = 53)

- Ok. **15–17 cyfr znaczących** (`sys.float_info.dig` → `15`)
- Największy `float`: `1.7976931348623157e+308`; dalej `inf`. Istnieje też `nan` („nie liczba”)
- Standard IEEE 754 obowiązuje od **1985 r.** i działa w niemal każdym procesorze

Dlaczego `0.1 + 0.2 != 0.3`?

- Ułamek `1/10` w systemie dwójkowym ma **nieskończone rozwinięcie** (jak `1/3` w dziesiętnym)
- Komputer przechowuje najbliższą liczbę, jaką da się zapisać na 53 bitach:

```
format(0.1, ".20f")          # '0.10000000000000000555'
0.1 + 0.2                    # 0.30000000000000004
0.1 + 0.2 == 0.3             # False
```

Zasada. Liczb `float` nie porównujemy przez `==`. Używamy tolerancji:

```
import math
math.isclose(0.1 + 0.2, 0.3)   # True
```

Gdy liczy się każdy grosz

- Do pieniędzy `float` się nie nadaje: błędy zaokrągleń kumulują się w księgowości
- Rozwiązanie: **arytmetyka dziesiętna** lub **ułamki**

```
from decimal import Decimal
from fractions import Fraction

Decimal("0.1") + Decimal("0.2")    # Decimal('0.3')
Fraction(1, 10) + Fraction(2, 10)  # Fraction(3, 10)
Decimal(0.1)                       # 0.1000000000000000055511151231257827...
```

- `Decimal` tworzymy **z tekstu**, nie z `float` (inaczej przenosimy błąd)
- Dziś w `zad3_waluty.py` wystarczy `float` z zaokrągleniem przy wyświetlaniu; w systemach finansowych – `Decimal`

Konwersje typów

- **Niejawna** (automatyczna): `int` + `float` → `float`, bo `float` „obejmuje” więcej wartości
- **Jawna**: wywołujemy typ jak funkcję

WYRAŻENIE	WYNIK	UWAGA
`int("42")`	`42`	
`int(3.9)`	`3`	obcięcie **w stronę zera**
`int(-3.9)`	`-3`	
`int("3.9")`	`ValueError`	tekst musi być liczbą całkowitą
`float("1,75")`	`ValueError`	separator dziesiętny to **kropka**
`bool("0")`	`True`	każdy niepusty tekst jest prawdą
`bool("")`	`False`	

Operatory i ich priorytet

Od najwyższego do najniższego:

OPERATOR	ZNACZENIE	ŁĄCZNOŚĆ
`**`	potęgowanie	**prawostronna**
`-x`	minus jednoargumentowy	
`* / // %`	mnożenie, dzielenie, dzielenie całkowite, reszta	lewostronna
`+ -`	dodawanie, odejmowanie	lewostronna

```
-2 ** 2       # -4, bo najpierw 2 ** 2
(-2) ** 2     # 4
2 ** 3 ** 2   # 512, bo 2 ** (3 ** 2)
7 / 7         # 1.0 – dzielenie / zawsze daje float
```

W razie wątpliwości: **nawiasy**. Kod ma być czytelny, nie sprytny.

Dzielenie całkowite i reszta

> **Twierdzenie (dzielenie z resztą).** Dla liczb całkowitych `a` i `b ≠ 0`:
> `a == (a // b) * b + a % b`

- `//` zaokrągla **w dół** (do $-\infty$), więc reszta `%` ma **znak dzielnika**

`A`	`B`	`A // B`	`A % B`
7	2	3	1
−7	2	−4	1
7	−2	−4	−1

- Zastosowania: parzystość (`n % 2 == 0`), ostatnia cyfra (`n % 10`), czas i zegar:

```
seconds = 3725
print(seconds // 3600, seconds % 3600 // 60, seconds % 60)   # 1 2 5
```

Zaokrąglanie

- `round()` stosuje **zaokrąglanie bankierskie** (*round half to even*), domyślne w IEEE 754: przy remisie wybiera liczbę parzystą, żeby błędy nie przesuwały sumy w jedną stronę

```
round(2.5)        # 2
round(3.5)        # 4
round(2.675, 2)   # 2.67 – bo 2.675 to naprawdę 2.67499999...
```

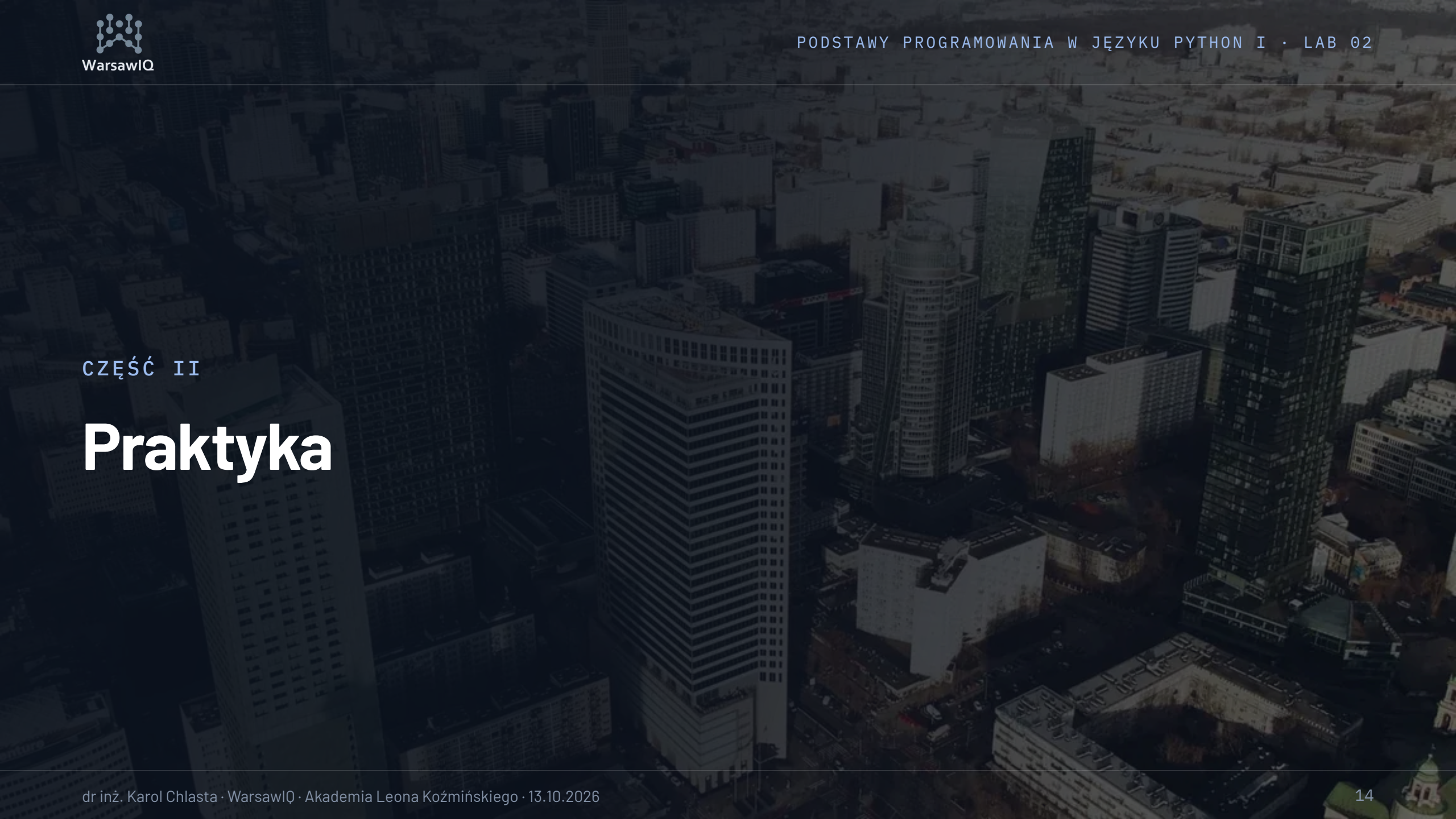
- **Zaokrąglenie ≠ formatowanie:**
  - `round(x, 2)` zmienia **wartość**
  - `f"{x:.2f}"` zmienia tylko **wyświetlanie**
- Testy w labie szukają liczb w wyjściu z tolerancją, więc wystarczy formatowanie

Przykład z 2.675 jest opisany w dokumentacji funkcji `round()`.

Przypisanie

```
x = 10
x += 5              # x = x + 5   → 15 (działa też -=, *=, /=, //=, %=, **=)
a, b = 1, 2         # przypisanie wielokrotne
a, b = b, a         # zamiana wartości bez zmiennej pomocniczej
VAT_RATE = 0.23     # stała – konwencja PEP 8, Python jej nie pilnuje
```

- Prawa strona `=` jest **obliczana najpierw**, dopiero potem nazwy są przypinane
- Dlatego `a, b = b, a + b` to cały krok ciągu Fibonacciego w jednej linii



CZĘŚĆ II

Praktyka

Typy i konwersje w praktyce

```
age = 20              # int
height = 1.75         # float
name = "Anna"         # str

int("42")       # 42
float("3.5")    # 3.5
str(7)          # "7"
type(height)    # <class 'float'>
```

Dane z `input()` to zawsze `str` → **najpierw konwersja, potem obliczenia**.

Ciekawostki: błędy liczbowe w realnym świecie 🤯

- **Patriot, Dhahran, 25.02.1991:** zegar systemu liczył czas w dziesiątych częściach sekundy, a `0,1` zapisane binarnie na 24 bitach było obcięte. Po ok. 100 h pracy błąd wynosił **0,34 s** i rakieta przechwytująca nie trafiła w pocisk Scud. Zginęło 28 żołnierzy (raport GAO/IMTEC-92-26)
- **Ariane 5, lot 501, 4.06.1996:** ok. 37 s po starcie konwersja prędkości z 64-bitowego `float` na 16-bitową liczbę całkowitą przekroczyła zakres. System nawigacji się wyłączył, a rakieta uległa samozniszczeniu (raport komisji J.-L. Lionsa)
- Wniosek: **typ danych i konwersja to decyzje inżynierskie**, nie szczegół techniczny

Ciekawostki: jednostki i Fibonacci

- **Mars Climate Orbiter** (NASA, 23.09.1999) przepadł przy Marsie – oprogramowanie podawało impuls w funtach-sekundach, a system NASA oczekiwał niutonosekund (różnica 4,45×). Misja kosztowała **327,6 mln USD**
- Ciąg **Fibonacciego** wprowadził do matematyki europejskiej Leonardo z Pizy w *Liber Abaci* (1202) – jako zagadkę o rozmnażaniu królików
- Iloraz kolejnych wyrazów dąży do **złotej proporcji** $\varphi = (1 + \sqrt{5}) / 2 \approx 1{,}618$
- −40 °C = −40 °F – jedyna temperatura równa w obu skalach

Dzisiejsze zadania

	PLIK	ZADANIE
★	`zad1_kalkulator.py`	7 działań na dwóch liczbach
★	`zad2_temperatura.py`	°C ↔ °F
★	`zad3_waluty.py`	PLN → EUR, USD
★	`zad4_czas.py`	sekundy → `h min s`
☆	`zad5_fibonacci.py`	ciąg Fibonacciego

Sprawdzenie i oddanie

```
cd lab02_zmienne_typy
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 02"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- Po zajęciach – **0 pkt**, bez wyjątków
- Pamiętaj: **Sync fork** na początku zajęć, żeby pobrać nowy lab

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 2
- *Floating-Point Arithmetic: Issues and Limitations* – rozdział oficjalnego samouczka (*docs.python.org/3/tutorial/floatingpoint.html*)
- D. Goldberg, *What Every Computer Scientist Should Know About Floating-Point Arithmetic*, ACM Computing Surveys 23(1), 1991
- Dokumentacja modułów `decimal` i `fractions`

Następny lab (20.10): instrukcje warunkowe